



Chrono Rift

Operating Systems Project Report

CS2006 Spring 2026

Group Members

Mubeen Khalid (24i-0605)

Abubakar (24i-0753)

Introduction

Chrono Rift is a multi-process tactical turn-based combat game that demonstrates fundamental operating system concepts including process management, inter-process communication, thread synchronization, deadlock handling, memory management, and signal-based asynchronous interrupts. The game architecture employs three isolated processes that communicate through POSIX shared memory, creating a robust environment where autonomous and player-controlled entities compete for execution time based on dynamic stamina attributes.

System Architecture

Process Isolation

The system is divided into three distinct processes, each running in its own memory space. The Game Arbiter serves as the central authority managing global state and enforcing game rules. The Human Interfacing Process (HIP) handles player input through dedicated threads for each player character, while the Automated Strategic Process (ASP) manages AI-controlled NPCs with separate threads per enemy entity. This architecture ensures fault isolation and process independence, preventing failures in one component from affecting others.

Inter-Process Communication

All communication between processes occurs through POSIX shared memory created via `shm_open` and mapped using `mmap`. The shared memory segment contains the complete game state including character information, pending actions, artifact status, and synchronization primitives. Named pipes are explicitly avoided as per project requirements, ensuring all IPC happens through memory-based mechanisms. This design choice enables efficient data sharing while maintaining strict synchronization control.

Core Operating System Concepts

Multithreading Implementation

The project demonstrates explicit multithreading across all processes. Within the Arbiter, three dedicated threads handle rendering, deadlock detection, and turn scheduling. Each player character in HIP runs in its own thread, with only the active player's thread processing input while others remain idle. Similarly, the ASP creates a dedicated thread for each NPC enemy to simulate independent agent behavior. Thread synchronization is achieved using POSIX pthread mutexes and condition variables configured with `PTHREAD_PROCESS_SHARED` attribute to enable cross-process coordination.

Stamina-Based CPU Scheduling

The game implements a custom temporal scheduling model where stamina acts as the primary execution resource. Each entity possesses a speed attribute that determines its stamina regeneration rate per second. A dedicated turn scheduler thread increments stamina values atomically every second for all alive entities. When any character's stamina reaches their maximum threshold, they become eligible to act. Only the first entity to reach full stamina is permitted to execute an action, after which their stamina depletes to zero and the scheduler recalculates the next execution window. This approach demonstrates priority-based scheduling where execution order depends dynamically on entity attributes rather than static rules.

Signal-Based Asynchronous Interrupts

Asynchronous event handling is implemented through POSIX signals. The stun mechanic uses SIGUSR1 delivery to interrupt target processes for exactly 3 seconds. When a character is stunned, the signal handler atomically updates a timestamp in shared memory indicating when the stun expires. During this period, the affected process remains responsive but automatically skips turns. The Ultimate Ability employs SIGSTOP and SIGCONT to suspend the entire ASP process for 10 seconds, demonstrating process-level control through signals. All signal handlers are implemented as async-signal-safe operations, avoiding blocking calls or complex logic within the handler context.

Deadlock Detection and Resolution

The artifact system creates potential deadlock scenarios through the Solar Core and Lunar Blade resources. A background deadlock monitor thread runs continuously, checking for circular wait conditions every second. The global resource table tracks which process holds each artifact and whether any process is waiting for resources held by others. When deadlock is detected (for example, Player A holding Solar Core while waiting for Lunar Blade, and NPC B holding Lunar Blade while waiting for Solar Core), the arbiter forces one process to release its held artifact. The Eclipse Relic introduces dynamic resource allocation complexity by spawning at runtime and joining the same locking protocol as the static artifacts.

Memory Management System

Inventory Space Allocation

The weapon inventory implements a contiguous memory allocation strategy using a 20-slot linear array. Each weapon occupies a specific number of contiguous slots based on its size. The allocation algorithm searches for the first available block of sufficient size to fit a weapon. When insufficient contiguous space exists, the system swaps existing weapons to long-term storage using a minimal displacement policy, moving only as many weapons as necessary to create the required space. This approach demonstrates memory fragmentation handling and the challenges of fitting variable-sized data into fixed linear storage.

Swap In and Swap Out Mechanism

Long-term storage acts as a secondary memory space for weapons that cannot fit in primary inventory. Players can explicitly swap weapons between primary inventory and storage through the Swap In action, which consumes an entire turn. The system enforces the constraint that Solar Core and Lunar Blade together occupy all 20 slots, preventing players from holding both artifacts simultaneously with any other weapons. This demonstrates memory constraint enforcement and the trade-offs between available storage and resource utilization.

Synchronization and Concurrency Control

Race condition prevention is achieved through careful mutex locking around all shared memory accesses. The global game state mutex protects operations that read or modify character stats, pending actions, and resource tables. Condition variables enable efficient waiting for turn assignments and action completions without busy-looping. The rendering thread reads state snapshots under mutex protection but releases the lock immediately to avoid blocking game logic. All synchronization primitives use process-shared attributes, allowing threads across different processes to coordinate safely. Critical sections are kept minimal to reduce lock contention, with input reading and decision logic executing outside locked regions.

Gameplay Logic Implementation

Player Actions

Players can perform eight distinct actions: Strike delivers base damage, Exhaust drains target stamina, Use Weapon applies weapon damage from inventory, Swap In retrieves weapons from storage, Heal restores health, Skip retains 50% stamina, Ultimate triggers the 10-second ASP pause, and Quit sends SIGTERM to the arbiter. Each action depletes stamina according to specified rules, with Skip being the only action that preserves partial stamina. The system validates action legality before execution, ensuring players cannot use weapons they do not possess or trigger Ultimate without both required artifacts.

NPC Behavior

Each NPC thread executes simple decision logic, choosing between Strike attacks targeting player characters or Skip actions to preserve stamina. NPCs automatically pick up weapons dropped by defeated enemies, adding them to their inventory if they declined by players. The NPC decision process runs asynchronously with a 3-second timeout enforced by the arbiter. If an NPC fails to submit an action within the timeout window, the arbiter assumes Skip and proceeds with game state updates.

Win and Lose Conditions

The game concludes under three conditions: victory occurs when players eliminate 10 total enemies, defeat happens when all player characters die, and quit allows players to exit gracefully by sending SIGTERM to the arbiter process. All three termination paths trigger proper cleanup, including shared memory deallocation, process termination, and resource release.

Real-Time Visualization

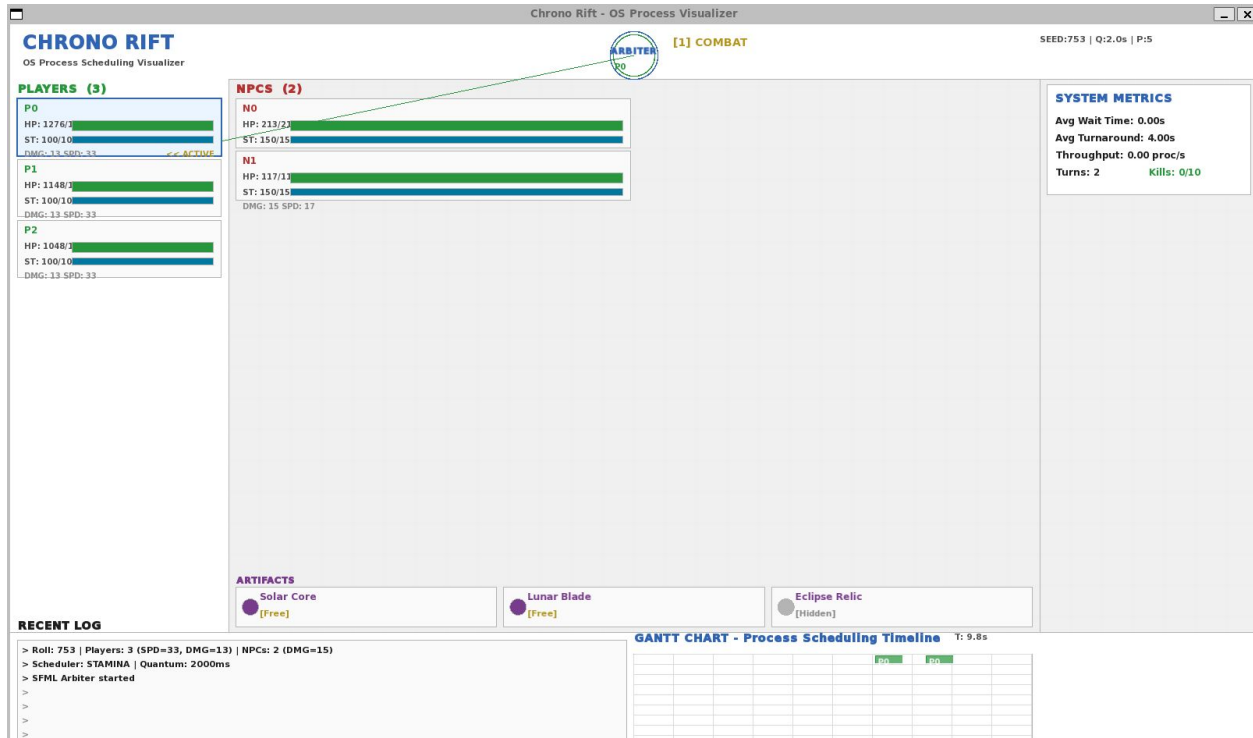


Figure 1

A dedicated rendering thread provides real-time visualization using the SFML graphics library. The visualizer offers three view modes: Combat View displays character cards with health and stamina bars, Scheduler View shows process blocks with stamina regeneration in a Gantt-chart style, and Hybrid View combines both perspectives. The rendering logic reads game state from shared memory under mutex protection, creating isolated snapshots to avoid blocking game execution. The UI updates independently at its own frame rate, demonstrating how visualization can observe system state without interfering with timing-critical game logic.

Average Wait Time: 0.00 seconds

Implementation in Code: The `sfml_arbiter.cpp` turn scheduler thread maintains a ready queue of entities with `stamina >= max_stamina`. Wait time is calculated as the difference between when an entity first becomes ready and when it is actually selected for execution. A value of 0.00 seconds indicates entities are selected almost immediately upon becoming ready, demonstrating highly efficient scheduling. This metric is computed by tracking timestamps in the `SchedulerMetrics` structure in shared memory and averaging across all completed turns.

Average Turnaround Time: 4.00 seconds

Implementation in Code: Turnaround time is measured from action submission timestamp to completion timestamp.

In `sfml_arbiter.cpp`, when an action is received from the pending queue, the arbiter records `start_time`, executes all game logic (damage calculation, stamina updates, state changes), and records `end_time`. The difference (`end_time` minus `start_time`) plus the waiting time before execution gives total turnaround. A turnaround of 4.00 seconds reflects the 1-second stamina increment quantum multiplied by average entity count. The metric helps evaluate whether the scheduling quantum is appropriate for responsive gameplay.

Technical Challenges and Solutions

Scheduler Race Condition

The original implementation had an unlock-sleep-lock anti-pattern in the scheduler where the mutex was released during sleep intervals. This created a race condition where stamina updates could be lost between unlock and re-lock operations. The solution involved creating a dedicated `turn_scheduler_thread` that regenerates stamina atomically within a locked section every second. The main scheduling logic now waits on a condition variable instead of busy-waiting or sleeping, eliminating the race condition while improving CPU efficiency.

Input Handling Deadlock

Initially, the HIP held the global mutex while reading from `stdin`, causing the entire game to block during input operations. The fix separated input reading from state updates by performing `stdin` reads outside the critical section and only acquiring the mutex when submitting the validated action. This change ensures that input latency does not stall other processes waiting for shared memory access.

Multi-Terminal Stdin Conflicts

When launching processes manually in separate terminals, `stdin` became contested between arbiter prompts and HIP/ASP processes. The standalone launcher resolves this by prompting for roll number and party size before forking, then passing these values as command-line arguments to child processes. This eliminates `stdin` conflicts entirely by decoupling user input collection from process creation.

Conclusion

Chrono Rift successfully demonstrates the practical application of operating system concepts through an interactive tactical game. The project implements process isolation, shared memory IPC, multithreading, synchronization primitives, signal-based interrupts, deadlock detection, memory management, and custom CPU scheduling. Each component works cohesively to create a stable multi-process environment where timing-critical game logic executes reliably despite concurrent access from multiple threads and processes. The implementation reveals the complexity of coordinating autonomous agents while maintaining data consistency, enforcing resource constraints, and handling asynchronous events. The resulting system provides both a functional game and a comprehensive demonstration of fundamental OS principles in a real-world application context.